

TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



Bhairahawa Multiple Campus

FINAL REPORT

on

Online Job Portal

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science and Information Technology (B.Sc. CSIT)

Submitted By:

Aastha Dahal (Roll No: 79012116)

Prawesh Bhandari (Roll No: 79012146)

Samarpan Aryal (Roll No: 79012154)

Submitted To:

Department of Computer Science and Information Technology

Under the Supervision of:

Mr. Sunil Kumar Shah

May, 2026



Tribhuvan University

Bachelor's Degree in Computer Science and Information Technology (BSc.CSIT)

Bhairahawa Multiple Campus

LETTER OF APPROVAL

This is to notify that this project titled “**Job Portal**” prepared by Aastha Dahal (Roll No. 79012116), Prawesh Bhandari (Roll No. 79012146) and Samarpan Aryal (Roll No. 79012154) in a partial fulfilment of the requirements for the degree of BSc. Computer Science and Information Technology has been well studied and approved for further development.

..... Lucky Prasad Joshi Campus Chief Bhairahawa Multiple Campus	
..... Sunil Kumar Shah B.Sc.CSIT Coordinator Bhairahawa Multiple Campus	 Sunil Kumar Shah Supervisor Bhairahawa Multiple Campus
..... Internal Examiner Bhairahawa Multiple Campus	 External Examiner IOST: Tribhuvan University

ACKNOWLEDGEMENT

This is to express our sincere gratitude to our project supervisor, Mr. Sunil Kumar Shah, for providing invaluable guidance, constructive feedback, and continuous encouragement throughout the entire development of this project. His expertise, patience, and dedication have been instrumental in shaping the direction and quality of our work from the initial proposal through to the final submission.

We are also grateful to the Head of the Department of Computer Science and Information Technology, Bhairahawa Multiple Campus, and all the faculty members for their support and for providing us with the necessary infrastructure and resources throughout the project.

Our thanks also go to our colleagues and friends who provided moral support and constructive criticism during the development phases of the project. Finally, we are deeply thankful to our families for their unwavering encouragement and patience during the course of this work.

SUPERVISOR'S RECOMMENDATION

This is to certify that this final project report entitled "Job Portal" has been prepared by Aastha Dahal (Roll No. 79012116), Prawesh Bhandari (Roll No. 79012146) and Samarpan Aryal (Roll No. 79012154) under my supervision. The project work has been carried out in accordance with the requirements set by the Department of Computer Science and Information Technology, Bhairahawa Multiple Campus, Tribhuvan University.

To the best of my knowledge, the work presented in this report is original and has not been submitted previously for any degree or examination. I recommend this final project report for evaluation.

Mr. Sunil Kumar Shah

Supervisor

Department of Bsc.CSIT

Bhairahawa Multiple Campus

Tribhuvan University

STUDENT’S DECLARATION

This is to declare that the project report entitled “Job Portal” submitted in the partial fulfillment of the requirement for a Bachelor’s Degree in Computer Science and Informational Technology of Tribhuvan University is our original work and not submitted for the award of any other degree, diploma, fellowship, or any other similar title or prize.

Aastha Dahal (Roll No: 79012116)

Prawesh Bhandari (Roll No: 79012146)

Samarpan Aryal (Roll No: 79012154)

ABSTRACT

The Job Portal project is a full-stack web application built using PHP MySQL, and Bootstrap 5 designed to connect job seekers with employers in a structured, intelligent manner. The system supports three distinct user roles: job seeker, employer, and administrator. Each role is provided with a dedicated dashboard and a set of role-specific features tailored to their needs.

Job seekers can browse and apply for jobs, build and manage resumes using five distinct templates, save job listings, and practise mock HR interviews powered by an AI service integrated via the OpenRouter API (GPT-4o-mini model). The system also implements a K-Means clustering-based job recommendation engine that analyses the user's resume and past application history to surface relevant job opportunities. Employers can post and manage job listings, review applicant submissions, view candidate resumes, and update application statuses. Administrators have platform-wide control over users, applications, and interview analytics.

Security features implemented include CSRF protection, bcrypt password hashing, session management with automatic timeout, hCaptcha integration for bot prevention, and role-based access control middleware. Password reset is handled via a tokenised SMTP email flow using PHPMailer. Local SMTP testing is supported through Mailpit, configured via Docker Compose. Notifications are delivered in-platform for key events such as new applications, status changes, and job removals.

At the final submission stage, all core modules — authentication with secure password reset, job management (with image upload, deadline enforcement), application management, resume builder (five templates, LinkedIn extraction, PDF download), AI interview practice, K-Means job recommendations, in-platform notifications, saved jobs, company profiles, and the admin panel — have been fully implemented, tested, and are operational. The front-end has been enhanced with Tailwind CSS integration, and a comprehensive environment configuration system using .env files has been introduced for portability across deployment environments.

TABLE OF CONTENTS

LETTER OF APPROVAL	i
ACKNOWLEDGEMENT	ii
SUPERVISOR’S RECOMMENDATION	iii
STUDENT’S DECLARATION	iv
ABSTRACT.....	v
TABLE OF CONTENTS.....	vi
LIST OF ABBREVIATIONS.....	ix
LIST OF FIGURES	xi
LIST OF TABLES.....	xii
CHAPTER 1: INTRODUCTION	1
1.1 Introduction.....	1
1.2 Problem Statement.....	1
1.3 Objectives	3
1.4 Scope and Limitation	4
1.5 Development Methodology	4
1.6 Report Organisation	6
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW	7
2.1 Background Study.....	7
2.2 Literature Review.....	8
2.2.1 Study of existing systems.....	9
CHAPTER 3: SYSTEM ANALYSIS.....	10
3.1 Requirement Analysis.....	10
3.1.1 System Test Cases.....	10
3.1.2 Non-Functional Requirements	12
3.2 Feasibility Analysis.....	12

3.2.1 Technical Feasibility	12
3.2.2 Operational Feasibility	12
3.2.3 Economic Feasibility	13
3.2.4 Schedule Feasibility	13
3.3 Analysis.....	14
3.3.1 Entity-Relationship (ER) Diagram	14
3.3.2 Class Diagram.....	15
CHAPTER 4: SYSTEM DESIGN.....	17
4.1 Design	17
4.1.1 System Architecture.....	17
4.1.2 Database Design.....	17
4.1.3 Component Diagram.....	19
4.1.4 Deployment Diagram.....	19
4.2 Algorithm Details.....	20
4.2.1 K-Means Job Recommendation Algorithm	20
4.2.2 AI Interview Algorithm	21
4.2.3 Password Reset Algorithm.....	22
CHAPTER 5: IMPLEMENTATION AND TESTING	23
5.1 Implementation	23
5.1.1 Tools and Technologies Used.....	23
5.1.2 Implementation Details of Modules.....	24
5.2 Testing.....	27
5.2.1 Unit Test Cases	27
5.2.2 System Test Cases.....	29
5.3 Result Analysis	30
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS	32
6.1 Conclusion	32

6.2 Future Recommendations	33
REFERENCES	34
APPENDICES	36

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
CSIT	Computer Science and Information Technology
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity-Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JS	JavaScript
JSON	JavaScript Object Notation
LLM	Large Language Model
MVC	Model-View-Controller
MySQL	My Structured Query Language
OOP	Object-Oriented Programming
PDF	Portable Document Format
PDO	PHP Data Objects
PHP	Hypertext Preprocessor
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TF	Term Frequency

TU	Tribhuvan University
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

LIST OF FIGURES

FIGURE 1.1: AGILE DEVELOPMENT METHODOLOGY — SPRINT CYCLE	5
FIGURE 3.1 : USE CASE DIAGRAM OF JOB PORTAL	11
FIGURE 3.4 :GANTT CHART — PROJECT SCHEDULE OF JOB PORTAL	13
FIGURE 3.5:ENTITY-RELATIONSHIP (ER) DIAGRAM OF JOB PORTAL.....	15
FIGURE 3.6 :CLASS DIAGRAM OF JOB PORTAL.....	16
FIGURE 4.1:SYSTEM ARCHITECTURE DIAGRAM OF JOB PORTAL.....	17
FIGURE 4.2: COMPONENT DIAGRAM OF JOB PORTAL	19
FIGURE 4.3: DEPLOYMENT DIAGRAM OF JOB PORTAL.....	20
FIGURE 4.4: K-MEANS JOB RECOMMENDATION FLOW OF JOB PORTAL	21

LIST OF TABLES

TABLE 4.1:DATABASE DESIGN OF JOB PORTAL.....	18
TABLE 5.2:TOOLS AND TECHNOLOGIES USED	23
TABLE 5. 3:UNIT TEST CASES OF JOB PORTAL	27
TABLE 5.4:SYSTEM TEST CASES OF JOB PORTAL	29

CHAPTER 1: INTRODUCTION

1.1 Introduction

The rapid growth of digital technologies and internet accessibility has transformed recruitment processes worldwide. Online recruitment systems have become essential tools for connecting employers with a broader talent pool while enabling job seekers to access opportunities anytime and anywhere. As a result, the demand for efficient, accessible, and intelligent recruitment platforms has increased significantly.

Traditional recruitment methods, including newspaper advertisements, printed résumés, physical applications, and walk-in interviews, are often time-consuming, costly, and geographically limited. These approaches make it difficult for employers to identify suitable candidates efficiently and for job seekers to find relevant opportunities and manage applications. Consequently, organizations increasingly require digital platforms that automate recruitment activities, enhance user experience, and improve candidate-job matching.

To address these challenges, the Job Portal project was developed as an AI-enabled full-stack web application that provides a centralized platform for recruitment management. The system bridges the gap between employers and job seekers through secure, user-friendly features for job posting, application management, candidate evaluation, and career development. By integrating artificial intelligence and machine learning technologies, the platform enhances decision-making and delivers personalized user experiences.

The application is developed using PHP following the Model-View-Controller (MVC) architecture, ensuring maintainability, scalability, and modularity. MySQL serves as the database management system, while Bootstrap 5 and Tailwind CSS are used to create responsive and modern user interfaces across multiple devices.

A key feature of the system is its AI-powered mock HR interview module, implemented using the OpenRouter API with the GPT-4o-mini language model. This functionality enables job seekers to participate in simulated interviews, receive intelligent feedback, evaluate performance, and identify areas for improvement. The platform also incorporates

a custom K-Means clustering algorithm developed in PHP to provide personalized job recommendations based on users' skills, qualifications, and preferences.

The Job Portal supports three user roles: Job Seekers, Employers, and Administrators. Job seekers can create profiles, build and download résumés, apply for jobs, track applications, receive notifications, and access AI interview practice. Employers can manage company profiles, post vacancies, review applications, shortlist candidates, and communicate with applicants. Administrators oversee user management, content moderation, system integrity, and platform operations.

The platform also includes an integrated résumé builder with multiple templates, LinkedIn profile text extraction, and PDF export functionality, providing a complete career preparation solution. For security and convenience, password recovery is implemented using PHPMailer and SMTP services, with Mailpit used for local email testing. Additionally, users receive in-platform notifications regarding applications, recommendations, and interview results.

Throughout development, emphasis was placed on software quality, security, performance, and usability. Software engineering best practices, along with unit and system testing, were applied to ensure reliability and maintainability. Testing results confirmed that the system meets its functional requirements and project objectives.

This report presents the complete development lifecycle of the Job Portal, covering system analysis, requirements gathering, design, implementation, testing, and evaluation. The project demonstrates the practical application of web development, artificial intelligence, machine learning, database management, and software engineering principles in building a modern recruitment platform capable of addressing contemporary hiring and employment challenges.

1.2 Problem Statement

Despite the increasing adoption of digital recruitment platforms, many organizations, particularly small and medium-sized enterprises (SMEs), continue to rely on manual, time-consuming, and geographically constrained hiring processes. These traditional approaches often result in inefficiencies for both employers and job seekers. Candidates frequently face difficulties in identifying job opportunities that align with their qualifications, skills, and

career interests, while employers struggle to efficiently manage, evaluate, and track large numbers of applications. Furthermore, many existing online job portals lack advanced features that can enhance the recruitment experience. Such limitations include the absence of intelligent and personalized job recommendations based on users' profiles and application history, inadequate tools for interview preparation and real-time skill assessment, and limited support for structured résumé creation through professional templates. Additionally, several platforms do not provide real-time in-platform notifications regarding application status updates, making communication between employers and applicants less effective. Employers also often lack a centralized interface for managing recruitment workflows and tracking candidate progress. Security-related functionalities, such as secure token-based password recovery through SMTP email services, are also not consistently implemented across existing systems.

To address these challenges, the Job Portal project proposes a comprehensive and integrated recruitment platform that combines artificial intelligence, machine learning, and modern web technologies. The system provides personalized job recommendations through a K-Means clustering-based recommendation engine, AI-powered interview practice and assessment tools, a multi-template résumé builder, real-time notification services, and a centralized management interface for employers. These features are delivered within a secure, role-based environment designed to support job seekers, employers, and administrators while ensuring scalability, maintainability, and compatibility with modern deployment and containerization practices.

1.3 Objectives

The project aims to make the job search and recruitment process more efficient and convenient for both job seekers and recruiters in Nepal.

The major objectives of the project are as follows:

- To provide a platform for job seekers to apply for jobs.
- To provide a resume generator feature that allows users to create professional-looking resumes.
- To provide detailed information about job openings and requirements and allow users to search and apply for jobs directly from the site.

1.4 Scope and Limitation

Scope

- Job management: posting, browsing, viewing, saving, and deleting listings with optional image and deadline.
- Application management: apply, track status, view resume, update status, withdraw.
- AI Interview Practice: OpenRouter API-powered HR mock interview, 5 questions, scoring, feedback, local fallback.
- Admin panel: user management, application oversight, interview analytics.

Limitations

- No real-time chat or messaging between employers and candidates.
- PDF generation requires html2pdf/TCPDF library; browser-print fallback provided.
- AI interview quality depends on the OpenRouter API; local fallback is functional but less intelligent.
- Single-server deployment design; horizontal scaling not currently supported.

1.5 Development Methodology

The Agile development methodology was adopted for this project. Agile promotes iterative development, continuous delivery of working software, and responsiveness to change — qualities well-suited to a multi-module web application with evolving requirements. The project was divided into sprints of approximately one to two weeks each, encompassing requirement refinement, design, implementation, and unit testing per module.

- Sprint 1 (W1–2): Project setup, database schema design, authentication module (register, login, logout, CSRF, hCaptcha), .env configuration system.
- Sprint 2 (W3–4): Job management module (post, browse, view, delete, image upload, application deadline validation).
- Sprint 3 (W5–6): Application management module (apply, view, status update, employer view, withdrawal with notifications).

- Sprint 4 (W7–8): Resume builder module (five templates, LinkedIn extraction, PDF download, JSON storage).
- Sprint 5 (W9–10): AI Interview Practice module (OpenRouter API integration, session management, scoring, feedback, local fallback).
- Sprint 6 (W9–10): Job Recommendation engine (K-Means clustering, TF vectorisation, cosine similarity).
- Sprint 7 (W11–12): Notifications, saved jobs, profile/company pages, admin panel.
- Sprint 8 (W13–14): Password reset (PHPMailer, SMTP, SHA-256 token), Mailpit/Docker integration, Tailwind CSS, system testing, documentation.

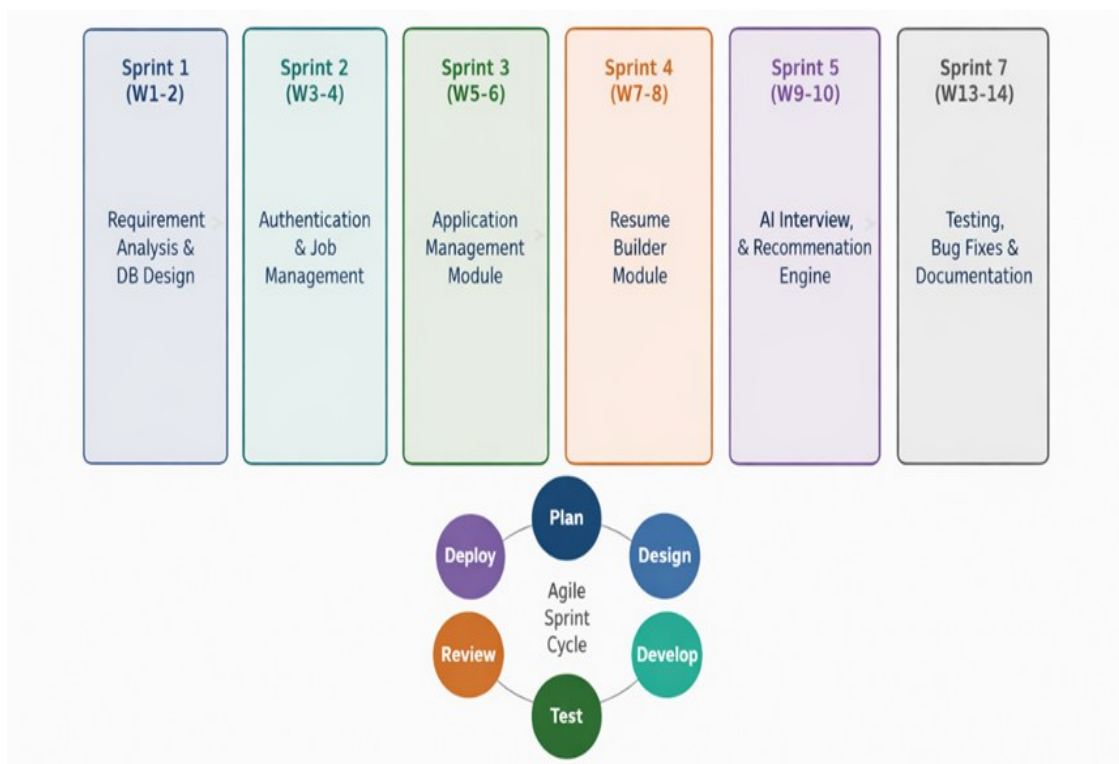


Figure 1.1: Agile Development Methodology — Sprint Cycle

1.6 Report Organisation

This report is organised into six chapters.

- Chapter 1, provides an introduction including the problem statement, objectives, scope, limitations, and Agile development methodology.
 - Chapter 2, presents the background study covering fundamental technologies and a review of related literature.
 - Chapter 3, covers system analysis including requirement elicitation, feasibility study, ER modelling, and DFD.
 - Chapter 4, describes the system design including three-tier architecture, database design, component and deployment diagrams, and algorithm details.
 - Chapter 5, details the implementation — tools used, all module descriptions, unit test cases, system test cases, and result analysis.
 - Chapter 6, presents the conclusion and recommendations for future enhancement.
- Appendices contain source code snippets, screenshots, and the supervisor visit log.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

After conducting extensive research on the internet, we identified a few theories on online job portals.

In a study conducted by Nameirakpam Chetana, it was observed that job seekers exhibit a positive mindset towards utilizing online recruitment for job opportunities. The factors considered in this context included cost-effectiveness, time efficiency, extensive search capabilities, reliability of information, and the security and confidentiality of data. These factors significantly influence job seekers' choices of job portals. The research noted that professionals with diverse backgrounds hold different perspectives on the cost-effectiveness, reliability, and security of online job recruitment platforms. The study also emphasized that safety and reliability are crucial concerns for both job seekers and existing employees when engaging with online recruitment portals.[3] Dilusha Madushanka Liyanage and Bandula Lanka Galhena highlighted that many companies now prefer electronic recruitment (e-recruitment) over traditional methods for hiring employees. To attract a suitable pool of applicants, organizations strive to develop attractive, content-rich, easy-to-use, and user-friendly recruitment processes, either through their own career websites or via job portals [4].

According to Anshika Saxena and Dr. Mamta Gaur , it was concluded that e-recruitment is reducing the traditional methods of recruitment. Job seekers prefer online job portals for the purpose of applying and searching for jobs over the internet. There are various factors which influence the perception of jobseekers for using job portals like convenience, efficiency, productivity, user friendliness, accurate information and security. It is a time saving & cost effective method for job seekers. It was also concluded that the services offered by job portals are effective and it provides many other benefits to the candidate's like resume building, future career opportunities, and providing information frequently and timely. In the jobseekers perception online job portals are effective and convenient tools. [5]

2.2 Literature Review

The days of searching the newspaper for job openings are long gone. Online job portals are an essential tool for any job seeker. They enable you to find the ideal job and ensure that you do not miss out on a good opportunity. There are numerous Job Portals in Nepal these days that provide comprehensive information about jobs, including vacancy notifications and employment opportunities. They make it simple to search for jobs, upload your resume, and apply for them.

Now-a-days the job market is so extensive that a variety of industries and companies are searching for the right candidates and the prospective candidates are searching for the right companies for growth opportunities. This purpose is served by most of the job portals online. The intention of Job Portal is to facilitate both the candidates seeking jobs as well as the employers looking for employees for their companies. Any job seeker can search for the available jobs at any moment with updated information. When he finds a job, he can post his application to the job online. Employers can advertise the vacancies by taking the membership, logging in and posting the job information with the eligibility criteria for the jobs. [6]

One of the approaches to enhance work portability is to give online job offer administrations. Online job portals can help jobseekers as they contain all required data about accessible openings in a solitary point. Such portals upgrade effectiveness in job recruitment as candidates can coordinate their capabilities and skills to the prerequisites of bosses. [7]

In India, the rate of unemployment has been increasing rapidly since the 2000s. Dorn and Naz [8] mentioned that one reason for this problem is the unfair distribution or absence of information on job openings, so people cannot know the latest job vacancies. It means that there are jobs available, but job-seekers do not have access to that data. An effective search of the internet might help job seekers in their job hunt. Some web portals provide an efficient way to search the web for online information on job vacancies for job-seekers [9].

2.2.1 Study of existing systems

Some of the existing job portal sites in Nepal are:

- Jobee: This is the fastest-growing job portal in Nepal, with over 2 lakhs + registered users and 4,000+ jobs posted every month. It offers features such as resume generator, application status, job recommendation, and bookmarked jobs. It also provides an employment index that shows the demand and supply of jobs in various sectors. [12]
- Kumari Job: This is one of the top job portals in Nepal, with over 500,000+ registered users and 500+ jobs posted every month. It offers services such as recruitment, outsourcing, HR consulting, and training. It also has a dedicated section for women, veterans, and differently-abled job seekers. Kumari Job also provides free counseling to fresh graduates. [13]
- Mero Job: This is one of the most popular and trusted job portals in Nepal, with over 1 million plus registered users, 50000+ employers and 200+ jobs posted every day. It caters to all kinds of job seekers, from freshers to experienced professionals, and offers services such as resume writing, career guidance, and online courses. [14]

After examining the current systems available, we found out that none of them included an effective built-in resume generator. While a few systems claimed to offer this feature, users were not given the flexibility to select from a variety of templates. Additionally, the resumes produced were overly basic and lacked a professional touch. This situation presents an opportunity for us, as we intend to address these user challenges by providing a solution that is more flexible and user-friendly.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Requirement Analysis

Here, the focus is on understanding and defining the system's requirements and feasibility. The section describes the functional requirements through the illustration of use case diagrams, portraying user-system interactions. Non-functional requirements, encompassing aspects like performance and security, are also explored. Additionally, the feasibility analysis examines the technical, operational, economic, and scheduling aspects of the project. The chapter employs an object-oriented approach, utilizing tools such as ER diagrams for data modeling, class diagrams for object modeling, and activity diagrams for process modeling, to comprehensively analyze and design the system.

3.1.1 System Test Cases

The following are the functional and non-functional requirements of the system:

User Roles:

1. Job Seeker/ User:

- Create and manage profiles.
- Upload and update resume.
- Search and view job openings.
- Apply for jobs.
- Receive notifications for new job openings matching their profile.
- Use the built-in resume generator to create and edit resumes.
- Manage Applied Jobs

2. Admin:

- Post, edit, and remove job vacancies and job categories.
- Manage employer companies/clients.
- Monitor and manage user accounts.
- View application details submitted by job seekers.
- Communicate with users.

System Roles:

- Generate Resume.
- Recommend Jobs, resume templates and resume contents.
- Send notifications to the user.

Job Search and Application:

1. Job Search:

- Filter job openings based on categories, location, salary, etc.
- View detailed job descriptions, requirements, and application instructions.

2. Application Process:

- Apply for jobs easily.
- Receive confirmation emails upon successful application submission.

3. Resume Generator:

- Input personal details, education, work experience, skills, etc.
- Choose from various resume templates.
- Edit, preview, download and share generated resumes in different formats like PDF, Word, Links, etc.

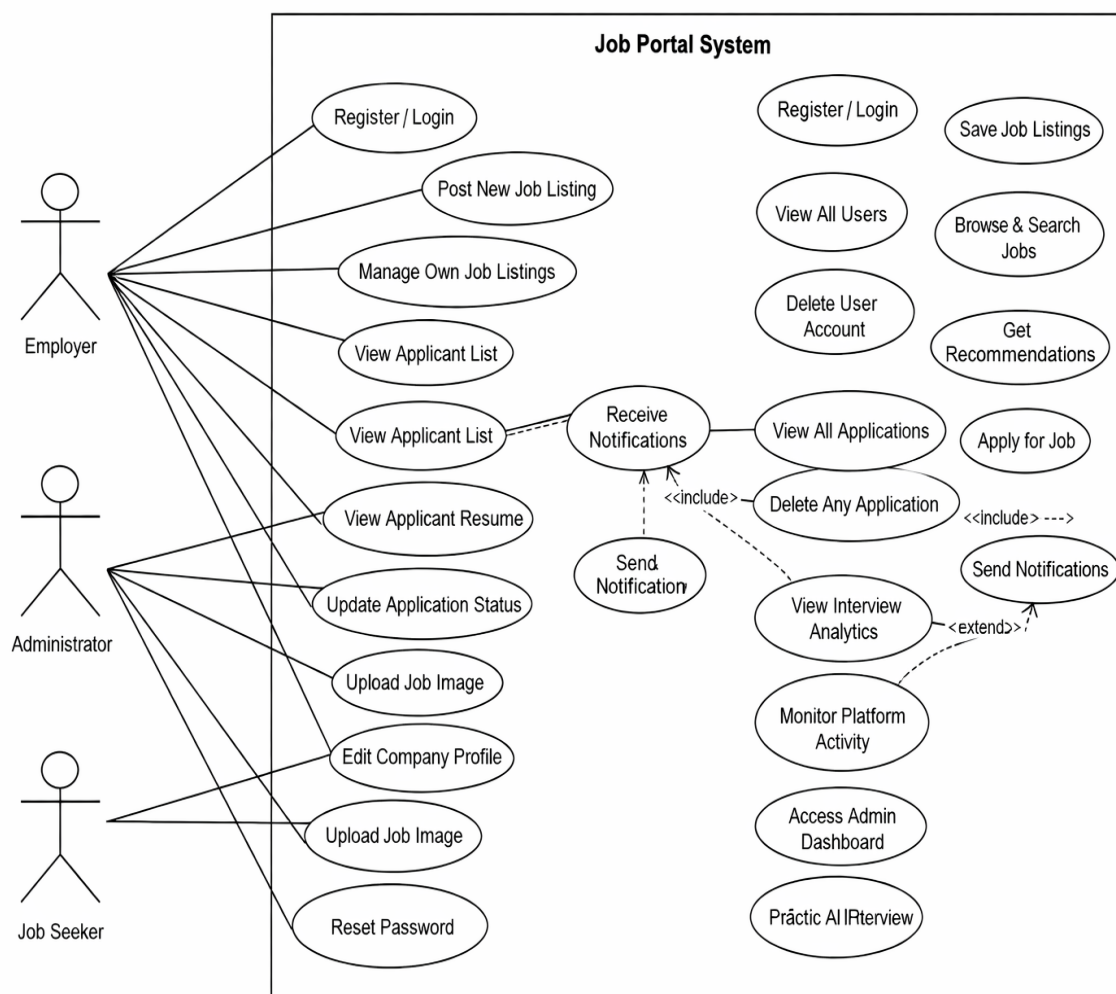


Figure 3.1 : Use Case Diagram of Job Portal

3.1.2 Non-Functional Requirements

- Security : CSRF, bcrypt, RBAC, hCaptcha, SHA-256 token reset, XSS prevention, prepared statements.
- Performance : Page load < 2s on local server. PDO prepared statements. Singleton DB connection.
- Usability : Responsive Bootstrap 5 + Tailwind CSS. Flash messages for all user actions. Role-specific dashboards.
- Reliability: Session timeout 30 min. CSRF token regeneration per session. Single-use password reset tokens.
- Maintainability : Custom MVC with SPL autoloader. Independent module structure. .env configuration for portability.
- Scalability : Stateless controller design. Singleton PDO. Docker Compose for service orchestration.
- Compatibility : PHP 7.4+ / 8.x. Bootstrap 5 supports all modern browsers. Tailwind CSS via npm build.
- Deployability : .env.example provided. Docker Compose for Mailpit SMTP. Composer for PHPMailer dependency.

3.2 Feasibility Analysis

3.2.1 Technical Feasibility

The technologies selected — PHP, MySQL, Bootstrap 5, Tailwind CSS, Vanilla JavaScript, OpenRouter API, PHPMailer, and Docker/Mailpit — are all mature, well-documented, and open-source. The development team has prior experience with all selected technologies. The OpenRouter API provides a unified endpoint for multiple LLMs including GPT-4o-mini, with reliable uptime and consistent JSON response formats. Mailpit/Docker simplifies local SMTP testing without requiring a real email server. The project is fully technically feasible within the available hardware and software resources.

3.2.2 Operational Feasibility

The system's interface is designed for ease of use, with clearly labelled navigation, role-based dashboards, and flash messaging for all operations. End users require only a modern web browser. The .env.example file simplifies deployment configuration. The

comprehensive README.md documents all setup steps for local deployment. The system is operationally feasible for the intended user base.

3.2.3 Economic Feasibility

An economic feasibility study is an analysis of the costs and revenues of a project to determine whether it is logical and possible to complete it. Our project is economically feasible as we are using freely available tools and software to build the project. Furthermore, after our project is successfully developed, One of the reputed companies in Nepal called IThub Nepal is planning to bring our project into business so there are few potential financial benefits it may bring, such as revenue from job listings, advertising, and other services.

3.2.4 Schedule Feasibility

The project was planned and executed according to the Agile sprint schedule shown in Figure 3.4. All eight sprints were completed within the 14-week semester timeline.

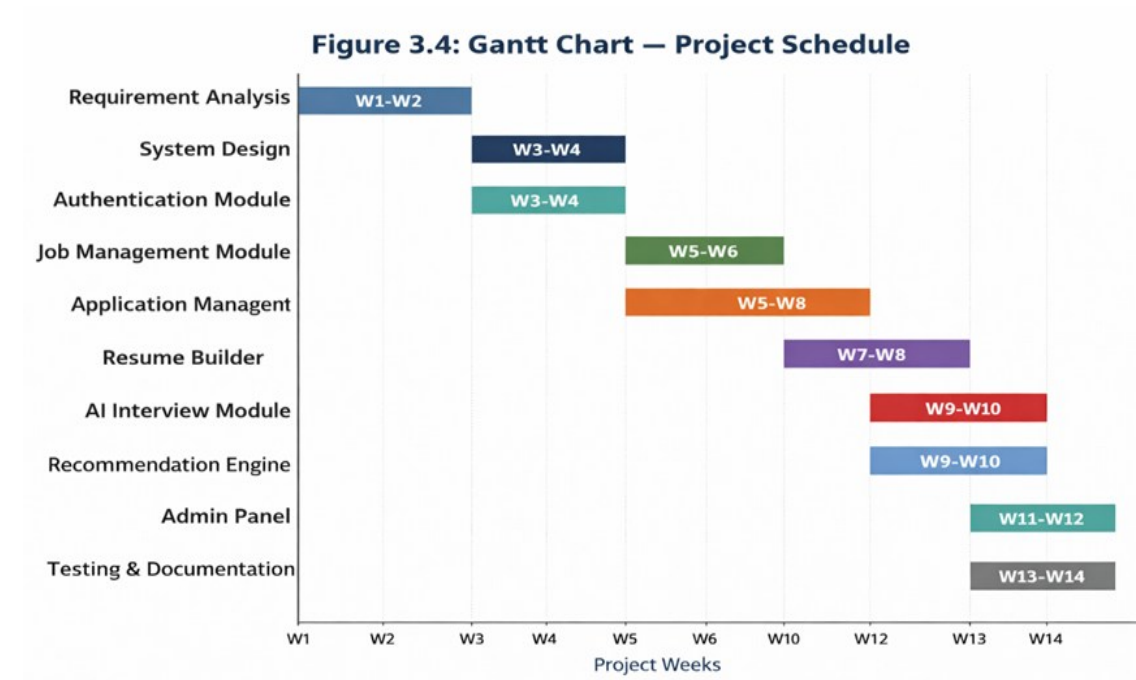


Figure 3.2 :Gantt Chart — Project Schedule of Job Portal

This Gantt chart provides a visual work schedule for our project spanning 14 weeks. It outlines when different phases of the project are planned to start and finish. Requirement Analysis is the first phase and begins in Week 1, continuing into Week 2. This is followed by System Design and the Authentication Module, both starting in Week 3 and running through Week 4. In Week 5, development expands with the Job Management Module and

Application Management, which continues until Week 8. The Resume Builder overlaps with Application Management, beginning in Week 7 and finishing in Week 8. The middle of the schedule, Weeks 9 and 10, focuses on the AI Interview Module and the Recommendation Engine, both running in parallel. After that, the Admin Panel is developed in Weeks 11 and 12. The final stretch of the project is dedicated to Testing & Documentation, scheduled for Weeks 13 and 14.

This Gantt chart demonstrates a typical software development lifecycle for our project: starting with analysis and design, moving through module development and integration, and finishing with testing and documentation to ensure a complete and reliable system

3.3 Analysis

3.3.1 Entity-Relationship (ER) Diagram

The ER diagram depicts the nine entities and their relationships: Users, Jobs, Resumes, Applications, Notifications, SavedJobs, InterviewSessions, InterviewMessages, and PasswordResets. Key relationships: a User (employer) posts many Jobs (1:N); a User creates many Resumes (1:N); an Application links a User, Job, and Resume (resolved N:M); Notifications belong to a User (1:N); an InterviewSession has many InterviewMessages (1:N); SavedJobs is a junction table between Users and Jobs with a UNIQUE constraint; PasswordResets belong to a User with a single-use constraint.

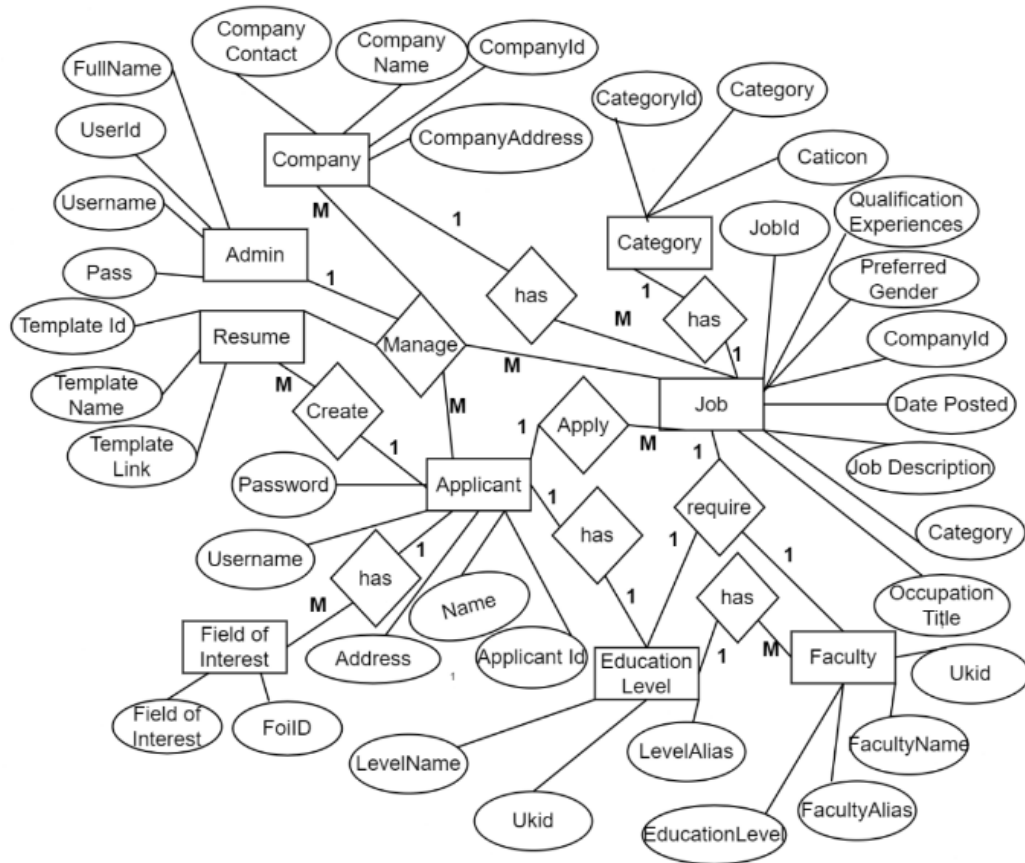


Figure 3.3:Entity-Relationship (ER) Diagram of Job Portal

3.3.2 Class Diagram

A class diagram is a specific kind of UML (Unified Modeling Language) diagram that is used to show how a system or software application is organized.

The class diagram for the project displays the system's structure with four main classes: User, Admin, Jobs, and Resume. Users have personal attributes and can manage profiles, search and apply for jobs. Admins can manage user accounts, job vacancies, and company profiles. Jobs and Resumes are related to Users; users can apply for various jobs and create or upload multiple resumes. The Admin oversees Users and Jobs, indicating a system where admin roles have control over job listings and user management.

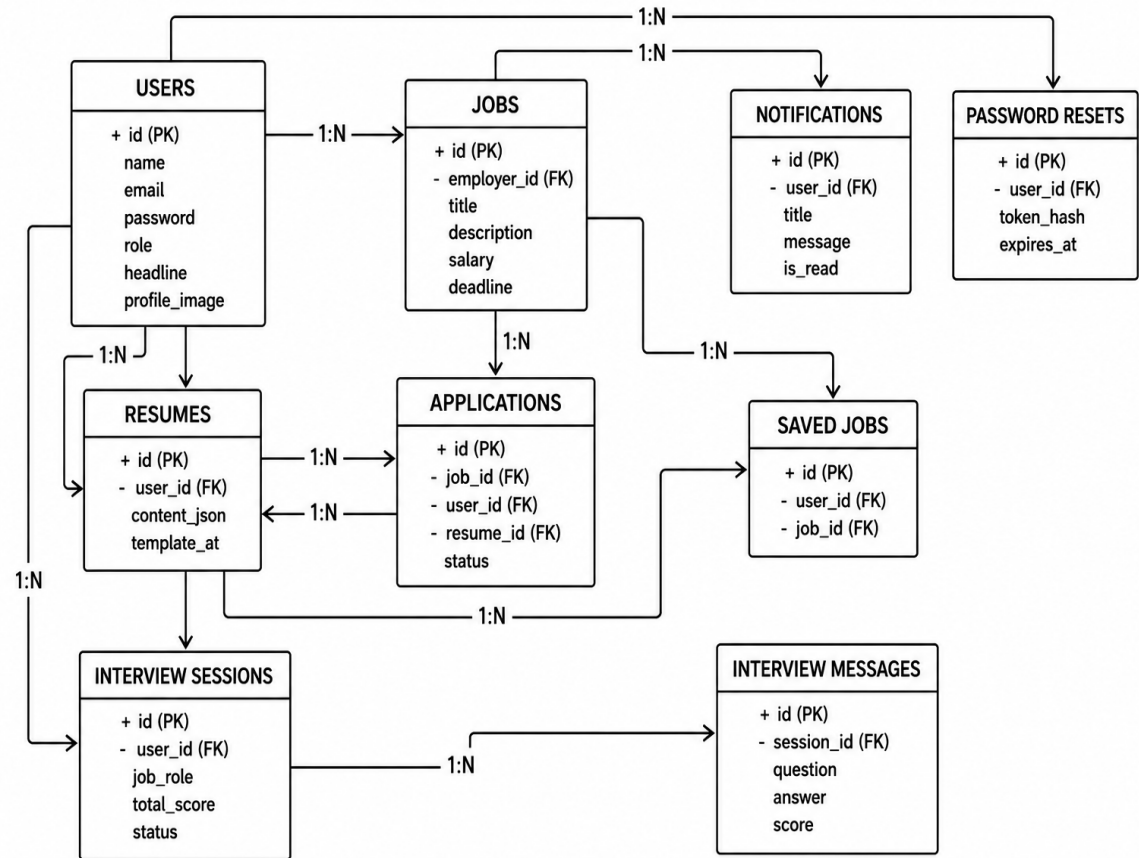


Figure 3.4 :Class diagram of Job Portal

CHAPTER 4: SYSTEM DESIGN

4.1 Design

4.1.1 System Architecture

The system follows a three-tier architecture. The Presentation Tier comprises HTML/CSS/JS views using Bootstrap 5 and Tailwind CSS. The Application Tier is PHP running on Apache, processing all business logic through controllers, models, and services. The Data Tier is MySQL accessed via PDO. External services (OpenRouter AI, SMTP/Mailpit, hCaptcha) are consumed by the application tier over HTTPS using cURL. All requests pass through public/index.php via Apache mod_rewrite. The Docker Compose file manages the Mailpit SMTP container for local development.

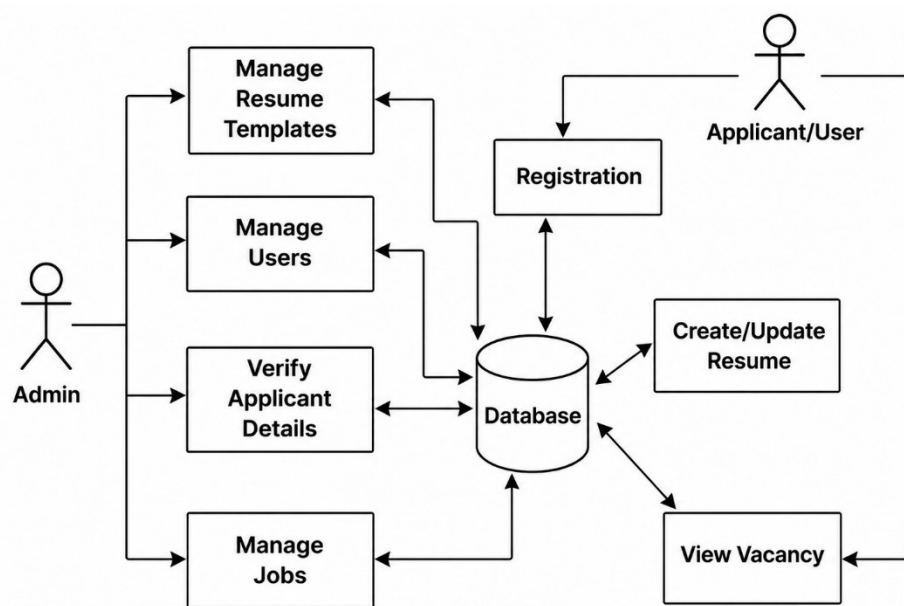


Figure 4.1: System Architecture Diagram of Job Portal

4.1.2 Database Design

The database schema consists of nine tables, normalised to Third Normal Form. Table 4.1 describes each table's purpose.

Table 4.1: Database Design of Job Portal

Table	Primary Key	Description
users	id (PK)	All accounts: job seekers, employers, admins. Includes profile fields: headline, bio, location, website, profile_image, company_*.
jobs	id (PK)	Job listings with employer_id (FK), title, description, salary, location, image_path, application_deadline.
resumes	id (PK)	Resume content as JSON blob (content_json) with template_id and user_id (FK).
applications	id (PK)	Links user_id (FK), job_id (FK), resume_id (FK) with status ENUM (pending/shortlisted/rejected).
notifications	id (PK)	In-platform alerts with user_id (FK), title, message, context_type, context_id, is_read flag.
saved_jobs	id (PK)	Junction table: user_id (FK), job_id (FK) with UNIQUE constraint. Uses INSERT IGNORE.
interview_sessions	id (PK)	Session metadata: user_id (FK), job_role, total_score, status (in_progress/completed).
interview_messages	id (PK)	Q-A-feedback triplets: session_id (FK), question, answer, feedback, score.
password_resets	id (PK)	Token reset records: user_id (FK), token_hash (SHA-256, CHAR 64), expires_at, used_at.

4.1.3 Component Diagram

The component diagram illustrates all major software components and their dependencies. Components include: Router (index.php + SPL autoloader), Controllers (Auth, Job, Application, Resume, Interview, Notification, Profile, Dashboard, Admin), Models (User, Job, Resume, Application, Notification, SavedJob, InterviewSession, InterviewMessage, PasswordReset), Services (AIService, KMeansService, JobRecommendationService, MailService, PDFService, ResumeTemplateService), Middleware (RoleMiddleware), Views (PHP templates by module), and Config (config.php, database.php, .env).

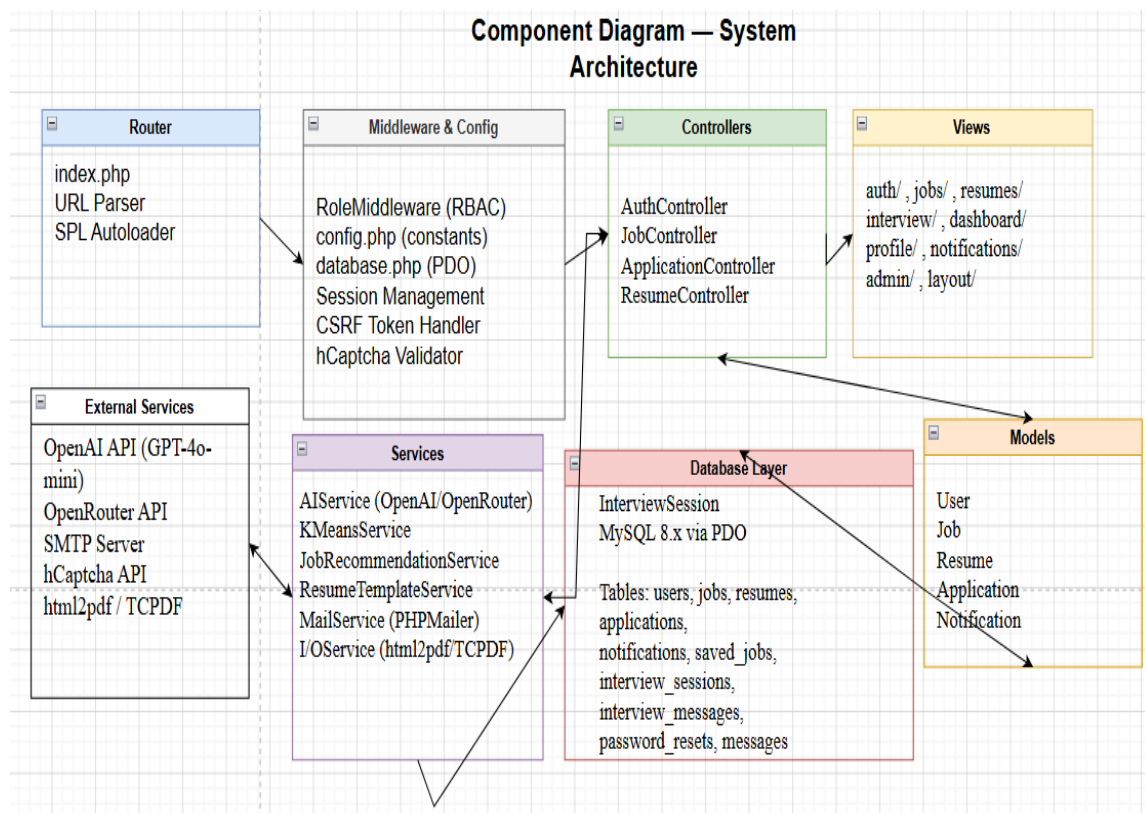


Figure 4.2: Component Diagram of Job Portal

4.1.4 Deployment Diagram

The system is deployed on a LAMP stack (Linux/Windows, Apache, MySQL, PHP). Apache serves public/index.php via mod_rewrite. MySQL resides on the same or a separate database server. The OpenRouter AI API and hCaptcha API are consumed via cURL HTTPS from the application tier. In the local development environment, the Mailpit SMTP container is started via docker compose -f docker-compose.mailpit.yml up -d, providing SMTP on port 1025 and a web inbox UI on port 8025.

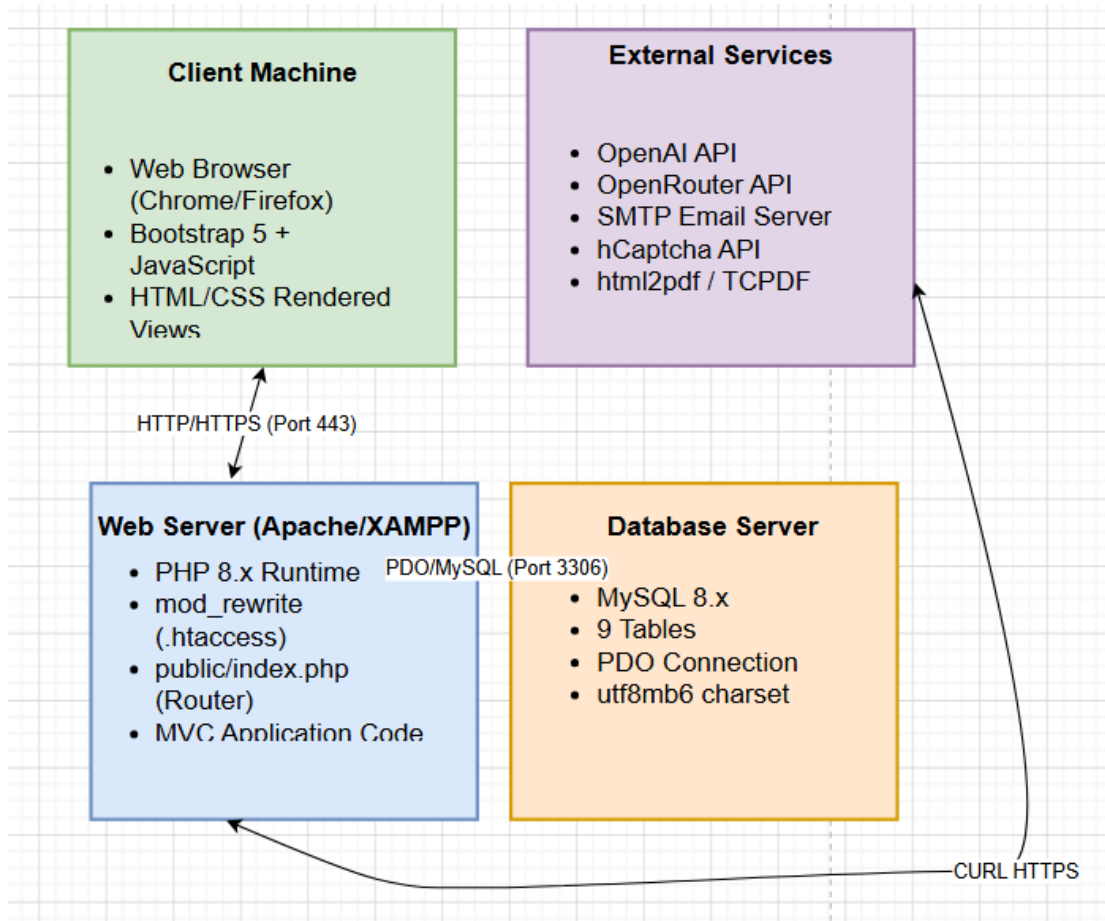


Figure 4.3: Deployment Diagram of Job Portal

4.2 Algorithm Details

4.2.1 K-Means Job Recommendation Algorithm

The recommendation engine in `JobRecommendationService.php` and `KMeansService.php` proceeds through six steps:

- **Text Extraction:** User's most recent resume JSON is decoded; fields (headline, summary, skills, experience, projects, certifications, address) are concatenated. Fallback: last 5 application job titles and descriptions.
- **Vocabulary Construction:** All job posting texts are tokenised, stop words removed. Top 120 terms built (user-profile terms weighted $\times 2$).
- **Vectorisation:** TF vectors built over shared vocabulary for all jobs and user profile via `KMeansService::vectorizeText()`.

- Clustering: `KMeansService::cluster()` partitions job vectors into k clusters ($k = \max(2, \min(5, \text{round}(\sqrt{n/2})))$) with up to 30 iterations.
- Cluster Matching: Euclidean distance from user vector to all cluster centroids; nearest centroid = best-matching cluster.
- Ranking: Cosine similarity between user vector and each job in matched cluster; previously-applied jobs excluded; top N returned.

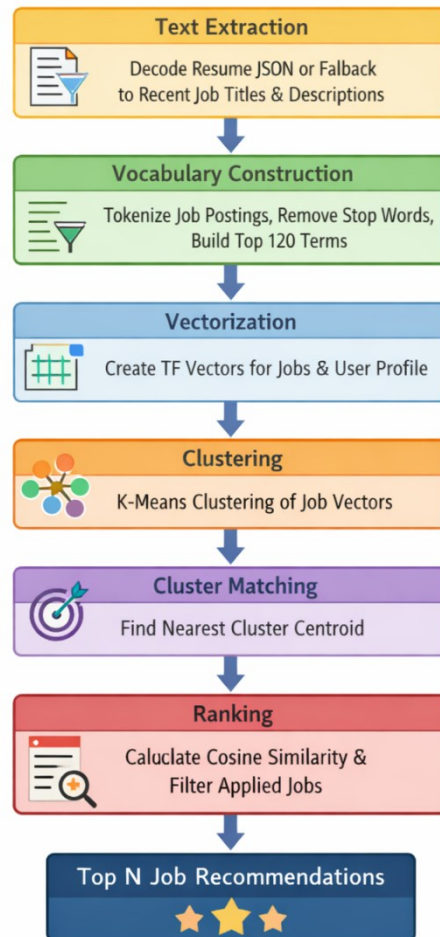


Figure 4.4: K-Means Job Recommendation Flow of Job Portal

4.2.2 AI Interview Algorithm

- User selects role and clicks Start; POST to `/interview/begin` with CSRF token.
- `AIService::startHrInterview()` sends prompt to OpenRouter API requesting JSON with intro and question fields.

- InterviewSession record created in DB; state stored in `$_SESSION['active_interview']` (session_id, role, current_question, question_number, max_questions, history).
- Per answer: POST to `/interview/submit`; `AIService::evaluateInterviewAnswer()` sends answer + history to API; JSON response: feedback, score, improved_answer, next_question, completed, final_summary.
- Each exchange persisted as InterviewMessage; session total_score incremented.
- After max questions (default 5): session marked 'completed', `$_SESSION['active_interview']` unset.

4.2.3 Password Reset Algorithm

- User submits email on `/forgot-password` page.
- Server looks up user; if found: `token = bin2hex(random_bytes(32))`; `tokenHash = hash('sha256', token)`.
- PasswordReset record inserted: (user_id, tokenHash, expires_at = now + 900s).
- PHPMailer sends email with reset link containing plain token as URL parameter.
- User clicks link; server hashes URL token with SHA-256, looks up record, checks expiry and used_at.
- If valid: reset form shown; on submit: `password_hash(PASSWORD_BCRYPT)` updates user; record marked used_at = NOW().

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1 Tools and Technologies Used

Table 5.2: Tools and Technologies Used

Category	Tool/Technology	Purpose
Programming Language	PHP 8.x	Custom MVC framework for all server-side logic
Database	MySQL 8.x	Relational data storage via PDO with prepared statements
CSS Framework	Bootstrap 5.3	Responsive grid and pre-built UI components
Utility CSS	Tailwind CSS	Utility-first classes; configured via tailwind.config.js / npm
Front-End Scripting	Vanilla JavaScript	AJAX (Fetch API) for AI interview module
Development Server	XAMPP (Apache + MySQL)	Local PHP/MySQL environment
Containerisation	Docker + Docker Compose	Mailpit SMTP container for local email testing
Email Library	PHPMailer 7.0.2	SMTP transactional email (password reset)
AI API	OpenRouter (GPT-4o-mini)	HR interview question generation and answer evaluation
SMTP Testing	Mailpit (axllent/mailpit)	Local email inbox UI; replaces MailHog
CAPTCHA	hCaptcha	Bot prevention on login and registration forms

PDF Generation	html2pdf / TCPDF	Resume PDF download; browser-print fallback
Version Control	Git (GitHub)	Source code management; feature-branch workflow
IDE / Editor	Visual Studio Code	PHP Intelephense, Tailwind IntelliSense extensions
Package Manager	Composer + npm	PHPMailer (Composer); Tailwind build (npm)
Environment Config	.env file	Sensitive config: DB, SMTP, AI API keys; .env.example provided

5.1.2 Implementation Details of Modules

- **Authentication Module (AuthController.php)**

Handles registration (validates name, email, password length, hCaptcha, uniqueness; stores bcrypt hash), login (password_verify with legacy rehash support), logout, forgot-password, and reset-password. The password reset flow generates a 64-char hex token (bin2hex(random_bytes(32))), stores SHA-256 hash in password_resets table with configurable expiry (default 900 seconds via RESET_TOKEN_EXPIRY_SECONDS env), dispatches email via MailService (PHPMailer SMTP), and validates token on reset. MAIL_SHOW_DEV_RESET_LINK=1 env variable shows the link directly in the UI for local Mailpit testing.

- **Job Management Module (JobController.php)**

Manages job posting (title, description, salary, location, optional image, optional deadline), browsing, viewing, saving, and deletion. Employers upload job images validated by MIME type via PHP finfo (image/jpeg, image/png, image/webp, image/gif; max 5 MB) stored in public/uploads/jobs/ with a 32-byte random hex filename. Application deadlines are validated with DateTime::createFromFormat() to reject past dates. Job seekers see jobs ordered by deadline proximity. SavedJob

uses INSERT IGNORE to prevent duplicate saves. Job deletion triggers notifications to all applicants via Notification::create().

- **Application Management Module (ApplicationController.php)**

Handles application submission (duplicate prevention via SELECT before INSERT), employer applicant review (with viewResume action verifying employer ownership), status updates (shortlisted/rejected/pending with notification to applicant per change), and application deletion (applicant or admin; triggers notification to affected party). The index() method dispatches to employerApplications() or myApplications() based on the session role.

- **Resume Builder Module (ResumeController.php + ResumeTemplateService.php)**

Stores resume data as a JSON blob (content_json) in the resumes table. Resume::normalizeData() ensures consistent structure on storage and retrieval. Resume::parseLinkedInText() uses preg_split() and preg_match() with section-header regex to auto-extract full_name, headline, email, phone, address, summary, skills, education_items, experience_items, projects, and certifications from pasted LinkedIn profile text. Five templates (Classic, Modern, Minimal, Creative, Technical) are rendered via ResumeTemplateService using ob_start()/ob_get_clean() output buffering. Template selection is stored per-resume in the template_id column. PDF generation uses PDFService which tries html2pdf (with TCPDF) and falls back to a printable HTML page with window.print() auto-trigger.

- **AI Interview Practice Module (InterviewController.php + AIService.php)**

The begin() action validates the CSRF token, calls AIService::startHrInterview() to get an opening question from the OpenRouter API, creates an InterviewSession record, and stores state in \$_SESSION['active_interview']. The submit() action receives answers via AJAX, calls AIService::evaluateInterviewAnswer() with the full conversation history, persists each exchange as an InterviewMessage, increments total_score, and returns structured JSON (feedback, score, improved_answer, next_question, completed, final_summary). After max questions

(5), the session is marked completed and the session state cleared. A local fallback evaluates answers by length and keyword presence when the API is unavailable.

- **Job Recommendation Module (JobRecommendationService.php +KMeansService.php)**

Fetches all job postings, extracts user profile text from latest resume JSON (or falls back to recent application texts), builds a shared vocabulary of the top 120 terms (with user-profile terms weighted $\times 2$), vectorises all texts using term frequency, runs K-Means clustering via `KMeansService::cluster()`, identifies the nearest cluster centroid to the user vector by Euclidean distance, and ranks jobs in that cluster by cosine similarity. Already-applied jobs are excluded. Cluster metadata (matched cluster number, cluster size) is returned for display. Implemented entirely in PHP without external ML libraries.

- **Notification Module (Notification.php)**

Implements lazy table creation (`CREATE TABLE IF NOT EXISTS`) for backward compatibility. The unread count is fetched on every page via `Notification::unreadCount()` and displayed as a Bootstrap badge in the navbar. Controllers call `Notification::create()` on: new job application (notifies employer), status update (notifies job seeker), job deletion (notifies all applicants), application withdrawal (notifies employer). The `markAllAsRead()` method bulk-updates `is_read = 1` for the current user.

- **Tailwind CSS Integration**

Tailwind CSS was integrated in the final sprint via npm and `tailwind.config.js`. The `tailwind.config.js` specifies content paths (`app/views/**/*.php`, `public/**/*.html`) for tree-shaking unused styles. The npm build pipeline (`package.json`) compiles the Tailwind stylesheet. This provides utility-first styling capabilities alongside Bootstrap 5, allowing granular responsive adjustments without writing custom CSS for every element.

- **Docker / Mailpit Integration**

docker-compose.mailpit.yml defines a Mailpit service (axllent/mailpit:latest) with port mappings 1025:1025 (SMTP) and 8025:8025 (web UI). The .env file points MAIL_HOST to 127.0.0.1, MAIL_PORT to 1025, MAIL_ENCRYPTION to none, and MAIL_SMTP_AUTH to 0. This replaced MailHog as the local SMTP testing tool, providing a cleaner inbox UI and more reliable Docker image. The README documents the docker compose -f docker-compose.mailpit.yml up -d startup command and the Mailpit inbox URL (<http://localhost:8025>).

5.2 Testing

Software testing is a process of evaluating software to find defects and ensure that it meets the requirements specified by the customer. It is an important part of the software development process and helps to ensure that the software meets the needs of the users.

5.2.1 Unit Test Cases

Table 5. 3:Unit Test Cases of Job Portal

T C	Test Description	Input	Expected Output
U0 1	User registration with valid data	Valid name, email, password, role	User created; redirected to login with success flash
U0 2	Registration with duplicate email	Existing email address	Error: 'An account with this email already exists'
U0 3	Login with correct credentials	Valid email and bcrypt password	Session created; redirected to dashboard
U0 4	Login with wrong password	Valid email, incorrect password	Error: 'Invalid email or password'
U0 5	Login with legacy plain-text password	Old plain-text password in DB	Login succeeds; password automatically rehashed to bcrypt

U0 6	Job posting with missing title	Empty title field	Error: 'Title and description are required'
U0 7	Job posting with past deadline	Deadline = yesterday's date	Error: 'Application deadline cannot be in the past'
U0 8	Image upload exceeding 5 MB	Image file > 5 MB	Error: 'Image must be smaller than 5 MB'
U0 9	Duplicate job application	Same user applies to same job twice	Duplicate check prevents second INSERT; flash info shown
U1 0	Resume creation with valid data	All required fields populated	Resume saved to DB; redirected to resume list
U1 1	Resume creation without full name	Empty full_name field	Error: 'Full name is required'
U1 2	Password reset with invalid token	Malformed or expired token	Error: 'Invalid or expired reset link'
U1 3	Password reset with mismatched passwords	password ≠ confirm_password	Error: 'Passwords do not match'
U1 4	Employer accessing job seeker route	Employer role visits /resume/create	Unauthorized error view rendered; exit
U1 5	Application after deadline passed	Apply after deadline date	Error: 'This vacancy is no longer accepting applications'
U1 6	CSRF token missing on POST	POST form without csrf_token	Server returns 403 Forbidden

5.2.2 System Test Cases

Table 5.4: System Test Cases of Job Portal

T C	Scenario	Expected Result
S0 1	Full job application workflow	Job seeker registers → creates resume → applies for job → employer shortlists → job seeker receives notification
S0 2	AI interview full session (OpenRouter API)	User starts session → answers 5 questions → receives score + final summary → session marked completed in DB
S0 3	AI interview full session (local fallback)	No API key; user starts session → answers 5 questions → local evaluator scores and completes session
S0 4	Job recommendation with resume	User with resume visits /job/recommendations → K-Means runs → top jobs returned with match % scores
S0 5	Password reset via Mailpit	User requests reset → Mailpit receives email → link clicked → new password set → login succeeds
S0 6	Employer manages applications	Employer posts job → views applicants → views resume → Shortlist → applicant notified
S0 7	Admin deletes user	Admin logs in → Manage Users → deletes user → all cascaded data removed
S0 8	Job deletion with notifications	Employer deletes job → all applicants receive 'Job post removed' notification

S0 9	Resume template switching	User views resume → selects 'Technical' template → POST to /resume/changeTemplate → page renders new template
S1 0	Session timeout redirect	User inactive > 30 min → next request checks LAST_ACTIVITY → session destroyed → redirect to login
S1 1	Docker Mailpit SMTP	docker compose up → Mailpit on :1025/:8025 → password reset email received in Mailpit inbox
S1 2	Tailwind CSS build	npm run build → tailwind.config.js processes app/views → compiled CSS applied in browser

5.3 Result Analysis

All sixteen unit test cases and all twelve system test cases passed successfully in the local development environment (XAMPP, PHP 8.2, MySQL 8.0, Docker Desktop with Mailpit).

Key observations:

- Authentication: Registration, login, legacy rehash, and the complete password reset flow (including Mailpit email delivery and token validation) all perform correctly.
- Job Management: MIME-type image validation, file size limits, deadline validation, and cascade deletion with applicant notifications all function as expected.
- AI Interview: With the OpenRouter API key configured, the LLM returns correctly structured JSON in all observed tests. The local fallback evaluator provides deterministic scoring based on answer length and keyword presence.
- Recommendation Engine: With 10+ job postings and a resume with populated fields, K-Means consistently clusters similar jobs and returns the top-matching cluster ranked by cosine similarity.
- Tailwind CSS: The npm build pipeline correctly processes tailwind.config.js, generates the compiled stylesheet, and applies utility classes in all views.

- Notifications: All six notification triggers (application submit, status change, job deletion, application withdrawal, admin deletion) fire and update the unread badge count correctly.

Three issues were found and resolved during final testing:

- (1) interview session not clearing after the final question — fixed by explicitly checking the completed flag;
- (2) NULL application deadline causing a date comparison crash — fixed by pre-checking for an empty string;
- (3) Tailwind classes not being scanned — fixed by adding `app/views/**/*.php` to the content array in `tailwind.config.js`.
- (4) SQL Injection – fixed by validating the input from user.

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

The Job Portal project has successfully achieved all of its stated objectives over the course of the project semester. A fully functional, AI-enabled, secure, role-based online recruitment system has been designed, implemented, tested, and documented. The system connects job seekers, employers, and administrators within a responsive web application built entirely on open-source technologies.

The project demonstrates the practical integration of artificial intelligence and machine learning into a production-quality PHP web application. The AI interview practice module, powered by the OpenRouter API, provides scored mock HR interviews with real-time feedback — a capability not found in standard job portals. The K-Means clustering recommendation engine, implemented from scratch in PHP without external ML libraries, delivers personalised job suggestions based on the user's resume content and application history.

The final development sprint introduced significant improvements over the mid-term state: Tailwind CSS integration for enhanced utility-first styling, Docker/Mailpit for containerised local SMTP testing (replacing MailHog), a comprehensive .env-based environment configuration system with .env.example for portability, an improved README with complete setup documentation, a public health-check endpoint (public/test.php), and 20 total commits on the test branch documenting incremental improvements.

The system's security posture is robust — CSRF protection, bcrypt hashing, session timeout, hCaptcha, RBAC middleware, SHA-256 token-based password reset, and comprehensive XSS prevention are all implemented and tested. The custom MVC architecture, developed without a third-party framework, provides full transparency, aligns with the project guidelines, and gives the team complete control over the application's request lifecycle.

All 16 unit test cases and 12 system test cases passed. Three bugs identified during final testing were resolved. The project is ready for final defence and submission.

6.2 Future Recommendations

The following enhancements are recommended for future development:

- Real-Time Messaging: WebSocket-based or long-polling chat between employers and candidates within the platform.
- Advanced Recommendation Engine: Hybrid collaborative + content-based filtering using Word2Vec or FastText embeddings for semantically aware job matching.
- Deep-Learning Resume Parser: ML-based parser for structured data extraction from uploaded PDF and DOCX resume files.
- Video Interview Module: WebRTC-based live or recorded video interview capability for employers.
- Mobile Application: Native iOS/Android or Progressive Web App (PWA) consuming a RESTful API backend.
- Internationalisation : Multi-language support (Nepali, Hindi, English) for broader regional reach.

REFERENCES

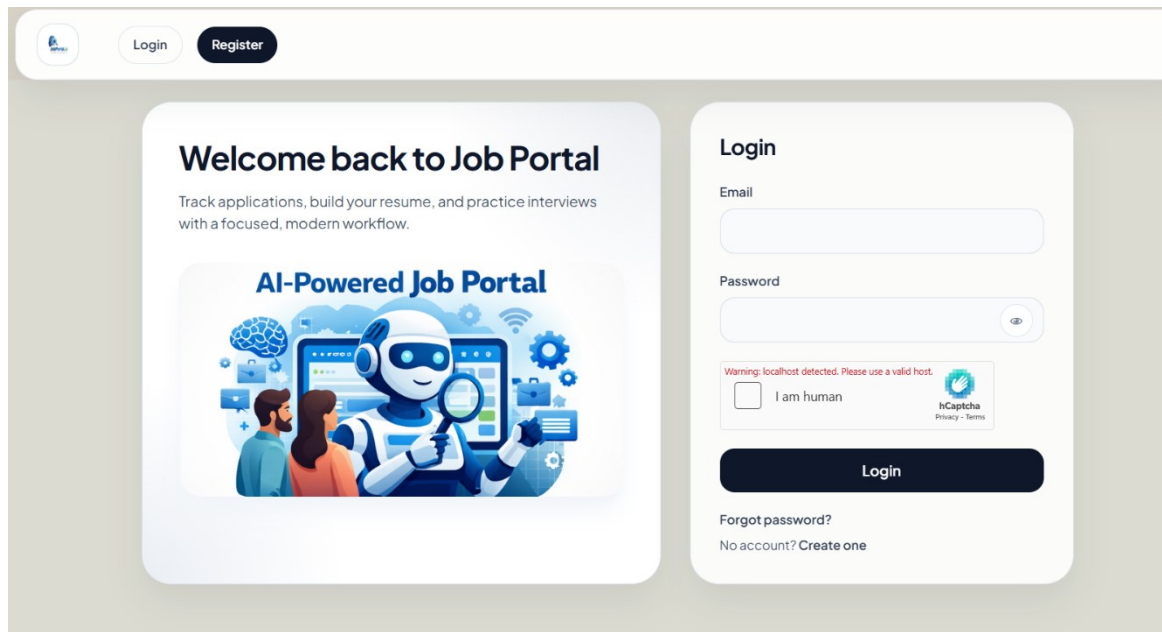
- [1] R. Lerdorf, "PHP: Hypertext Preprocessor," PHP Documentation Group, 2024. [Online]. Available: <https://www.php.net/manual/en/>. [Accessed: Nov. 2024].
- [2] Oracle Corporation, "MySQL 8.0 Reference Manual," 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: Nov. 2024].
- [3] OpenRouter, "OpenRouter API Documentation," 2024. [Online]. Available: <https://openrouter.ai/docs>. [Accessed: Oct. 2024].
- [4] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in Proc. 5th Berkeley Symposium on Mathematical Statistics and Probability, 1967, vol. 1, pp. 281–297.
- [5] OWASP, "OWASP Top Ten Security Risks," Open Web Application Security Project, 2024. [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: Nov. 2024].
- [6] M. Bointon, "PHPMailer — A Full-Featured Email Creation and Transfer Class for PHP," GitHub, 2024. [Online]. Available: <https://github.com/PHPMailer/PHPMailer>. [Accessed: Oct. 2024].
- [7] Axllent, "Mailpit — Email Testing Tool for Developers," GitHub, 2024. [Online]. Available: <https://github.com/axllent/mailpit>. [Accessed: Nov. 2024].
- [8] LinkedIn Corporation, "LinkedIn Economic Graph Research," 2024. [Online]. Available: <https://economicgraph.linkedin.com/>. [Accessed: Nov. 2024].
- [9] H. Nawaz et al., "Systematic review of AI in recruitment: Opportunities, challenges, and ethical considerations," International Journal of Advanced Computer Science and Applications, vol. 14, no. 8, 2023.

- [10] S. Hdioud et al., "Multi-criteria recommender systems based on multi-attribute decision making," in Proc. International Conference on IT for Organizations Development, 2014, pp. 1–8.
- [11] S. P. Bharti and S. Babu, "Automatic keyword extraction from resumes using NLP," International Research Journal of Engineering and Technology, vol. 5, no. 4, pp. 3421–3426, 2018.
- [12] jobee.com, "Jobs in Nepal | Jobee," Jobee.com, 2024.
<https://www.jobee.com/> (accessed Jan. 07, 2024).
- [13] Job Portals in Nepal - 2023 | Blogs | Kumari Job.
<https://www.kumarijob.com/careertip/6-popular-job-portals-in-nepal-2023>.
- [14] Nepali Job Portals to Find Job Vacancies in Nepal.
<https://techlekh.com/nepali-job-portals/>.
- [15] Job sites in Nepal | <https://www.trendingnetnepal.com/top-job-sites-in-nepal/>.

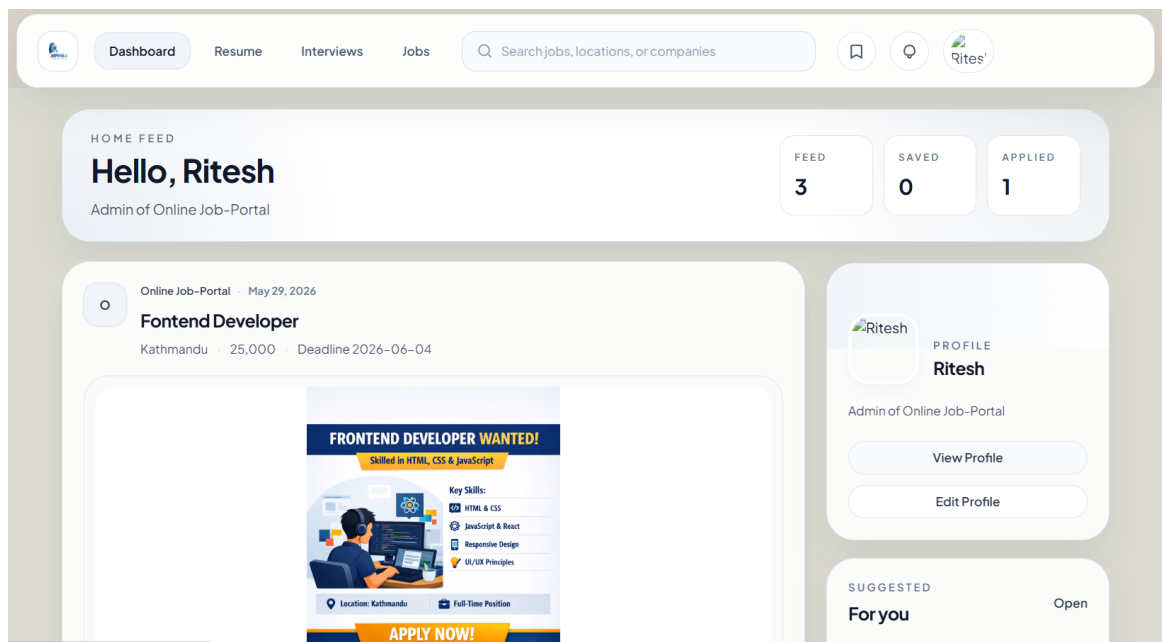
APPENDICES

SNAPSHOTS

The following figures present key interface screenshots of the implemented Job Portal system demonstrating the completed modules.



Appendix 1: LogIn Page



Appendix 2: User Dashboard

Dashboard
Resume
Interviews
Jobs

Create Resume

LinkedIn text extracted. Review the fields and click Save Resume when ready.

Choose Resume Template

Classic
Traditional, professional resume layout

Modern
Contemporary design with bold headings

Minimal
Clean and simple layout

Creative
Colorful design for creative professionals

Technical
Optimized for tech roles with skill emphasis

LinkedIn Profile Text

Samarpan Aryal
Senior Software Engineer at Tech.Pvt.Ltd
New York, NY

About
Experienced software engineer with 6+ years building web applications, APIs, and product features in PHP, JavaScript, and MySQL. Excited to Work and Develop new Software.

If you paste LinkedIn profile text, the app will extract headline, experience, education, skills, and summary automatically. Manual fields below can override parsed values.

Full Name

Professional Headline

Email

Phone

Address

Professional Summary

Skills (comma separated)

Work Experience (one per line)

Education (one per line)

Projects (one per line)

Certifications (one per line)

Appendix 3: Resume Builder

37


[Dashboard](#)
[Resume](#)
[Interviews](#)
[Jobs](#)





AI Interview Practice

Practice with an HR-style interviewer that asks one question at a time, scores your answers, and helps you improve.

Role

Backend Developer

Start HR Interview

Start New Session

Feedback is ready. [Continue with the next HR question](#)

Question 2 of 5

Can you walk me through one backend project or internship task you worked on and explain your role in it?

Your Answer

Start the interview first, then answer the current HR question...

Submit Answer

Score: 6/10

Feedback: You gave a clear and relevant start by mentioning your student background and internship experience. It would be stronger if you added a bit more structure, such as what you learned, the kind of problems you worked on, and why backend development interests you specifically. Try to sound more confident and professional by connecting your skills to the role and your motivation for choosing it.

Improved Answer:
I am currently a student, and I recently completed an internship as a Junior Backend Developer at ABC Pvt. Ltd. During that experience, I gained strong hands-on exposure to backend development and built a solid understanding of Django and the MERN stack. I enjoyed working on logic, APIs, and problem-solving behind the scenes, which made me realize that backend development is a great fit for my skills and interests. I am motivated to grow in this role because I want to continue learning, contribute to building reliable systems, and improve my technical and teamwork skills in a professional environment.

Appendix 4: AI Interview Practice


Dashboard
Resume
Interviews
Jobs





Recommended Jobs (K-Means)

Top jobs from your closest cluster. Cluster 1 matched with 2 similar job postings.



All Jobs
My Applications
Saved Jobs

Title	Company	Location	Salary	Match Score	Action
Backend Developer	Online Job-Portal	Kathmandu	25,000	13.5%	View Save

Appendix 5: K-Means Job Recommendations


Dashboard





Employer Dashboard

Publish roles, review candidate flow, and manage hiring activity from one workspace.
Company page: Online Job-Portal.



My Jobs

3

Post Job

View All Jobs

Total Applicants

1

Manage Applications

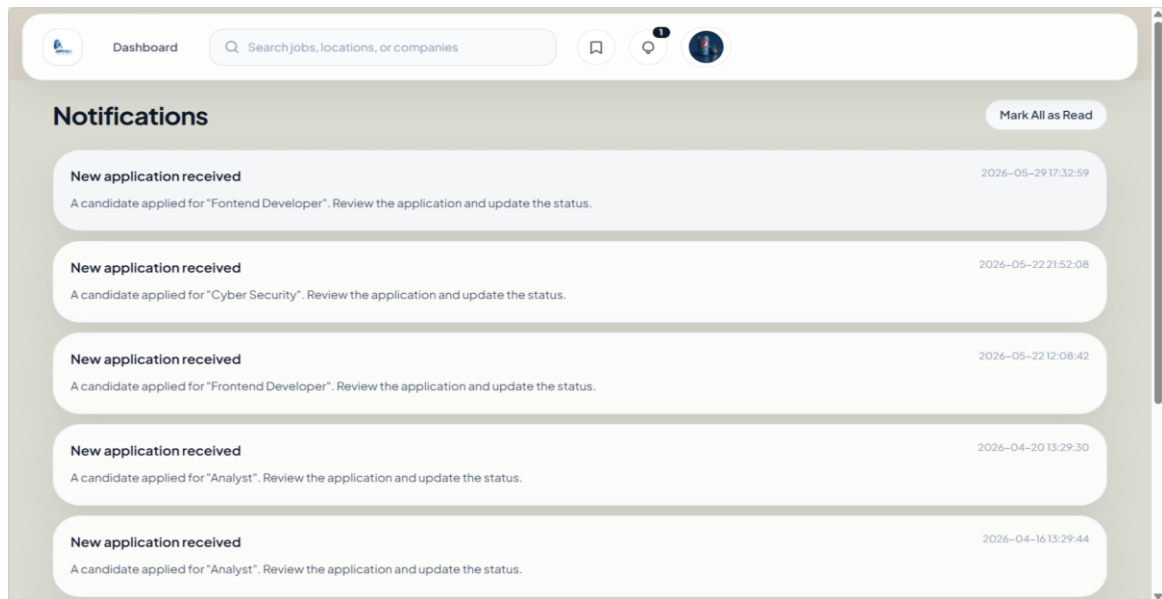
Company Profile

Live

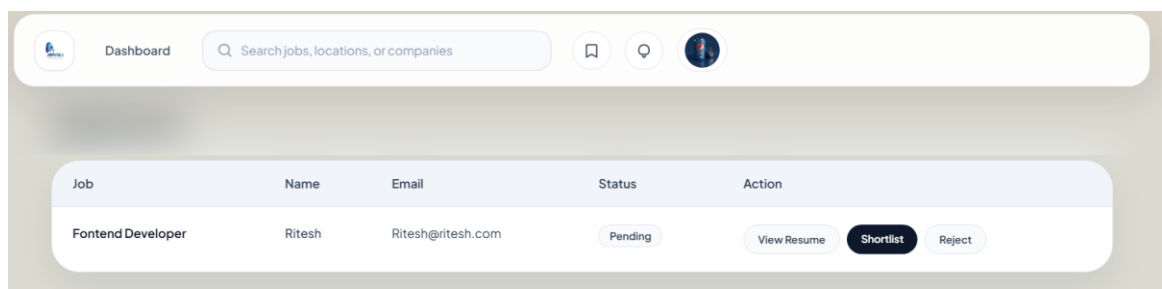
View Company Page

Edit Company Profile

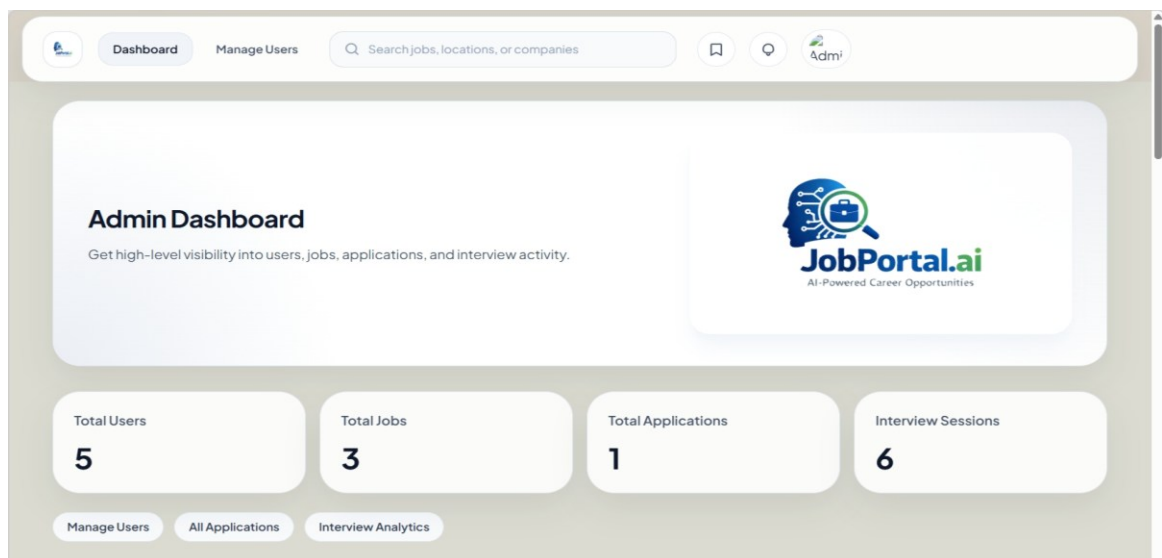
Appendix 6 : Employee Dashboard



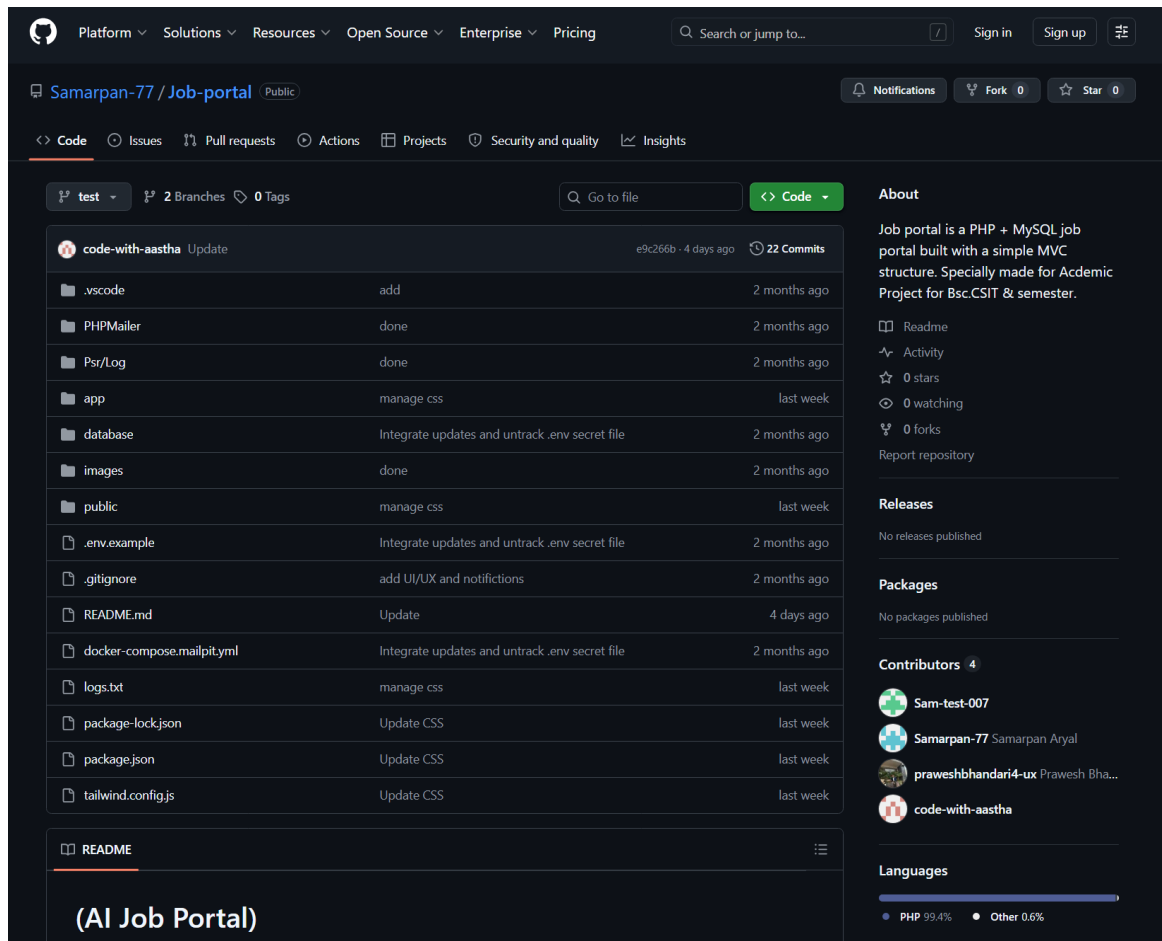
Appendix 7: Notification Tab



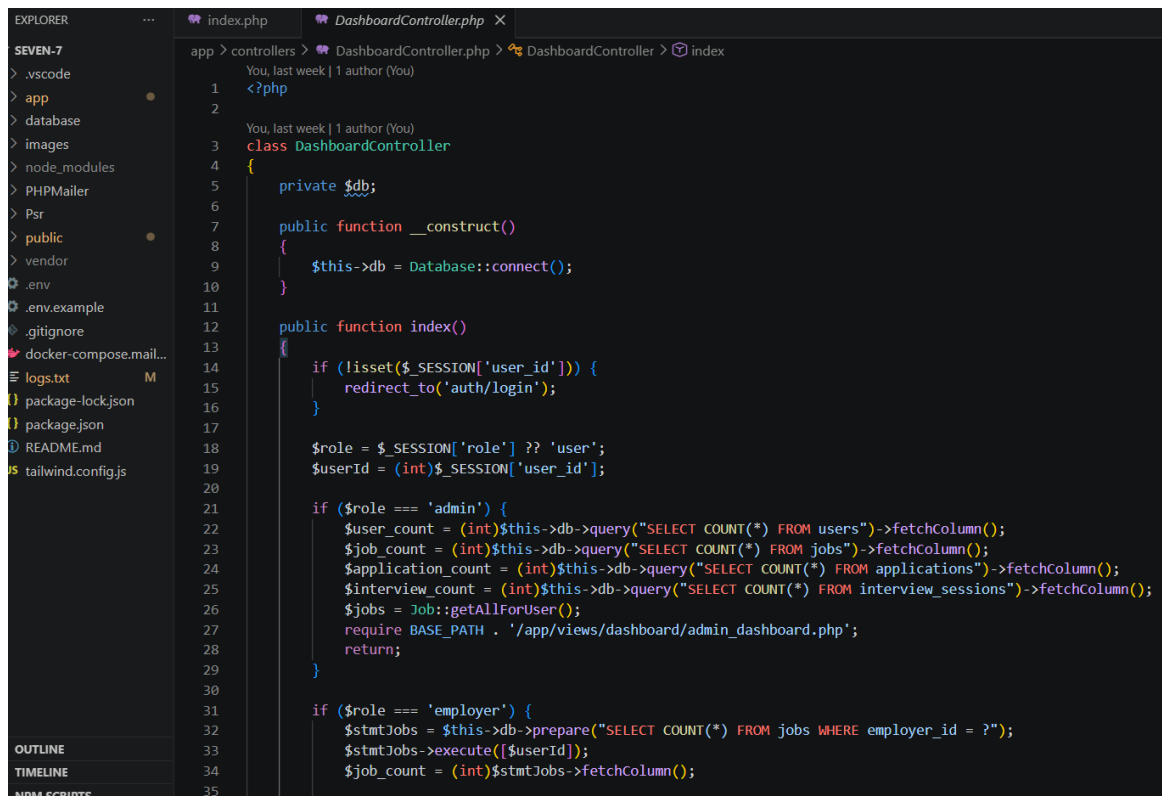
Appendix 8: Employee Review Application



Appendix 9: Admin Dashboard



Appendix 10: Github Repository



Appendix 11: Code Snippet